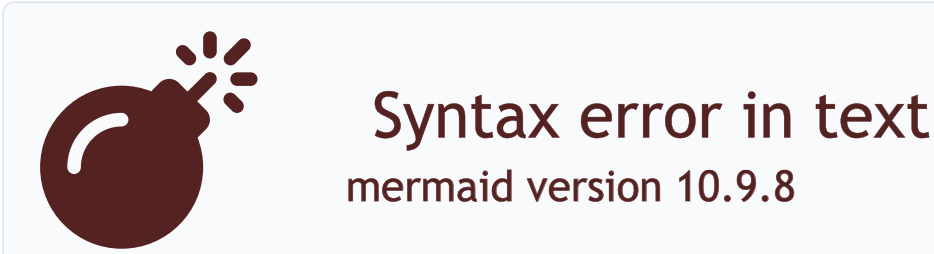
Submodule 36.10: PyTorch Neuron Interactive Notebook

1. Notebook Overview & File Reference

This submodule provides complete architectural documentation and detailed cell-by-cell analysis for the lecture demonstration notebook:

- **Source Notebook File**: `10_lecture_code_Neuron.ipynb`
- **Framework**: PyTorch (`torch` , `torch.nn`)



---

2. Cell-by-Cell Anatomical Breakdown

Cell 0: Package Imports

```
import torch
import torch.nn as nn
```

- **Technical Context**: Imports the core tensor computational engine (`torch`) and the high-level neural network abstraction module (`torch.nn`).

Cell 1: Reproducibility & Feature Vector Creation

```
# input
torch.manual_seed(42)
inputs = torch.tensor([1.0, 2.0, 3.0])
```

- **Tensor Properties**:
  - `dtype` : `torch.float32` (default floating-point precision)
  - `shape` : `torch.Size([3])`

- `device` : `'cpu'`

Cell 2: Instantiating the Linear Transformation Layer

```
neuron = nn.Linear(in_features=3, out_features=1)
```

- **Layer Anatomy**:
  - `neuron.weight` : Parameter tensor of shape `(1, 3)` . Initialized using Kaiming/He uniform initialization by default.
  - `neuron.bias` : Parameter tensor of shape `(1)` . Initialized from \mathcal{U}(-\frac{1}{\sqrt{ ext{in_features}}}, \frac{1}{\sqrt{ ext{in_features}}}).

Cell 3 & 4: Forward Pass Computation

```
output = neuron(inputs)
output
```

- Under the hood, invoking `neuron(inputs)` triggers `nn.Module.__call__` , which registers autograd hooks and executes `neuron.forward(inputs)` .
- Output is a 1-element tensor with gradient tracking enabled (`grad_fn=<ViewBackward0>`).

Cell 5 & 6: Mathematical Proof via Tensor Multiplication

```
# verify ... formula
neuron.weight @ inputs + neuron.bias
neuron.bias
```

- The `@` operator in PyTorch performs true matrix multiplication / dot product.
- Multiplying `(1, 3)` weight tensor with `(3,)` input vector yields a `(1,)` scalar, to which the `(1,)` bias scalar is broadcasted and added.

---

3. Summary of Learnings from the Notebook

1. A single artificial neuron in PyTorch is implemented as `nn.Linear(in_features, 1)` .
2. PyTorch manages weights and biases automatically as learnable `nn.Parameter` objects.
3. The forward pass of a neuron without activation is strictly equivalent to matrix multiplication plus bias addition.